UNIVERSIDADE FEDERAL DE LAVRAS - UFLA
CAMPUS SÃO SEBASTIÃO DO PARAÍSO
Bacharelado Interdisciplinar em Inovação, Ciência e Tecnologia

RELATÓRIO TÉCNICO: PROJETO 01

Tabelas Hash em Textos Reais: Análise de Desempenho e Dispersão

Milena Duarte Costa

Professor: Dr. Alysson Naves

São Sebastião do Paraíso - MG
24 de Abril de 2026

Sumário

1 Introdução **2**

2 Metodologia **2**

2.1 Regra de Normalização e Tokenização 2

2.2 Descrição das Funções Hash 2

3 Análise das Funções Hash **2**

4 Impacto do Fator de Carga (α) e do Tamanho M **3**

5 Análise Crítica dos Histogramas **4**

5.1 Distribuição em Tabela Pequena (M=97) 4

5.2 Distribuição em Tabela Ideal (M=997) 5

5.3 Validação em Língua Portuguesa 6

6 Comparação: Hash vs. Linear **6**

7 Análise da Função 'Ruim' (H_BadPrefix) **7**

8 Conclusão **7**

1 Introdução

Este relatório apresenta o estudo experimental sobre Tabelas Hash com encadeamento separado. O objetivo central foi avaliar a eficiência de diferentes funções de hashing no processamento de chaves extraídas de obras literárias, analisando métricas de colisão, fator de carga e custo de busca. A estrutura foi comparada com uma busca linear (baseline) para validar a redução da complexidade computacional em operações de dicionário.

2 Metodologia

2.1 Regra de Normalização e Tokenização

Para garantir a uniformidade dos resultados, o texto foi convertido integralmente para minúsculas. A tokenização foi realizada via Expressões Regulares (Regex) utilizando o padrão `r'[a-zà-ú]+'`. Esta regra captura palavras que contêm letras minúsculas e acentuações comuns às línguas portuguesa e inglesa, descartando números e pontuações. O conjunto de dados final trabalhado consistiu apenas nas palavras distintas de cada obra.

2.2 Descrição das Funções Hash

Foram implementadas funções com diferentes níveis de complexidade:

- **H1 (SumASCII):** Soma simples dos valores ordinais dos caracteres.
- **H2 (WeightedSum):** Soma ponderada pela posição do caractere, visando diferenciar anagramas.
- **H3 (Horner31):** Método polinomial de Horner com base 31, reduzindo colisões em strings longas.
- **H4 (XORShift):** Operações de bits (XOR e deslocamentos) para maximizar a dispersão dos bits dos caracteres.
- **H_BadPrefix:** Função de controle que utiliza apenas o primeiro caractere como índice, induzindo colisões intencionais.

3 Análise das Funções Hash

A escolha de uma função hash eficiente é crucial para o desempenho, impactando diretamente a distribuição dos elementos nos buckets e o número de colisões. Analisando os resultados obtidos com o texto *tale.txt* e $M = 997$:

Tabela 1: Métricas para o texto tale.txt (M=997)

Função Hash	**max_bucket**	**avg_comp_success**	**avg_comp_fail**
H1_SumASCII	48	10.9421	6.9583
H2_WeightedSum	30	7.3823	9.0978
H3_Horner31	21	6.0340	9.9973
H4_XORShift	23	5.9915	10.0163

Os valores de *max_bucket* indicam o tamanho do maior bucket. Um *max_bucket* menor sugere uma distribuição mais homogênea. Neste cenário, a função **H3_Horner31** apresentou o menor valor (21), seguida pela **H4_XORShift** (23). Isso prova que ambas proporcionam a distribuição mais uniforme, minimizando colisões severas e evitando que a busca degenere para um comportamento de lista linear.

4 Impacto do Fator de Carga (α) e do Tamanho M

O fator de carga ($\alpha = n/M$) é uma métrica fundamental que reflete a densidade da tabela. Analisando a função **H3_Horner31** com o texto *tale.txt*:

Tabela 2: Impacto da variação de M no desempenho (H3)

M	**Fator de Carga (α)**	**max_bucket**	**avg_comp_success**
97	102.4433	121	52.2363
100	99.3700	122	50.7521
997	9.9669	21	6.0340

Observa-se uma clara correlação inversa entre o tamanho da tabela M e o desempenho. Quando M é pequeno, o alto fator de carga aumenta a probabilidade de colisões. À medida que M aumenta para 997, α diminui drasticamente, resultando em uma redução substancial no *max_bucket* e no custo médio de busca, aproximando o desempenho de $O(1)$.

5 Análise Crítica dos Histogramas

5.1 Distribuição em Tabela Pequena (M=97)

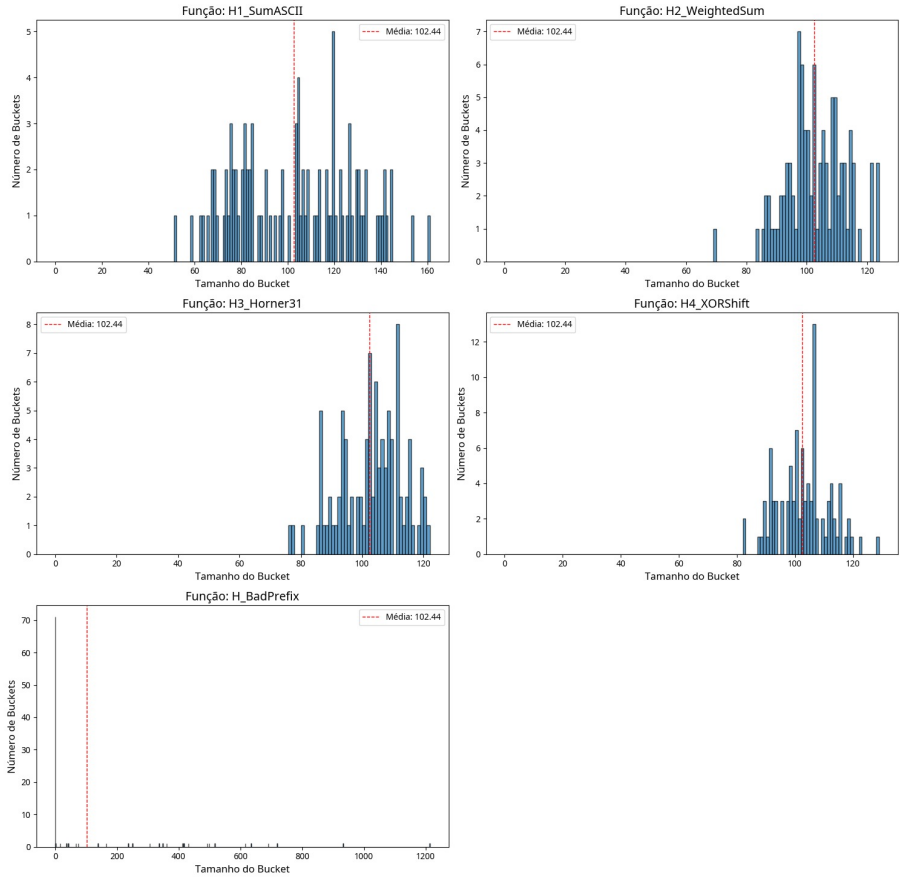

Figura 1: Distribuição para o texto tale.txt com M=97.

Análise: Esta imagem ilustra o cenário de congestionamento. Com o fator de carga elevado (≈ 102), as colisões são inevitáveis. As funções boas tentam equilibrar, mas a densidade é visível pela altura das barras em relação à média (linha vermelha).

5.2 Distribuição em Tabela Ideal (M=997)

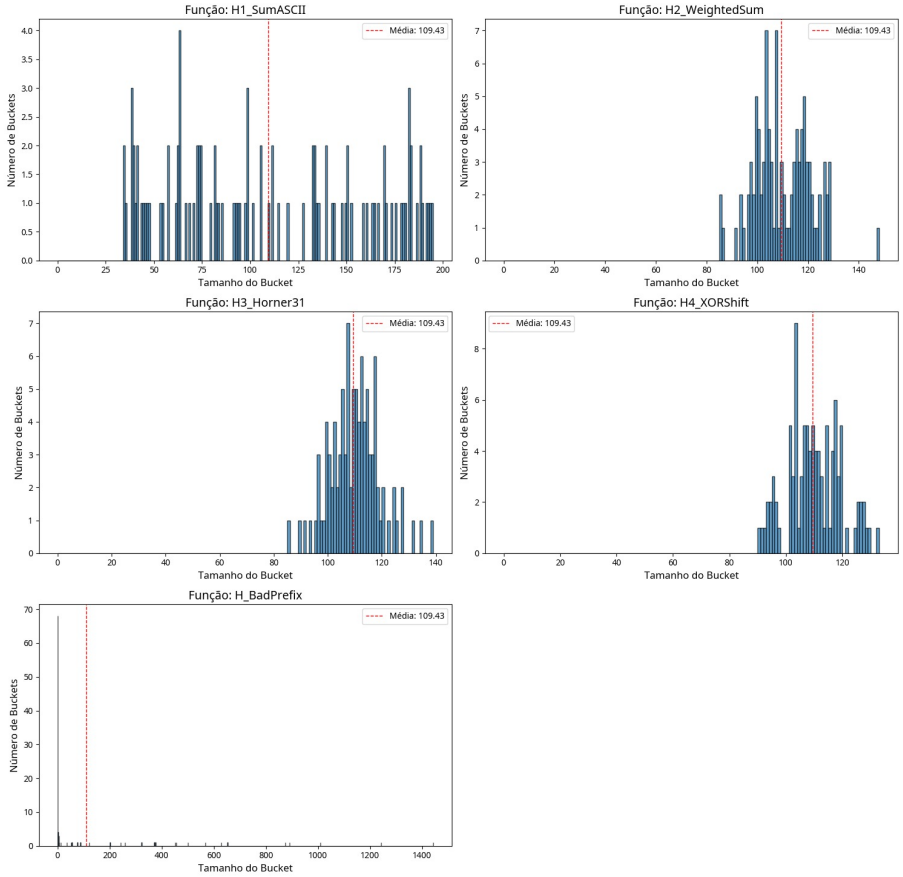

Figura 2: Distribuição otimizada para o texto tale.txt com M=997.

Análise: Aqui observamos a eficiência máxima. A dispersão das funções **H3** e **H4** é uniforme, mantendo os buckets com poucos elementos. Isso garante que o tempo de acesso seja quase constante, independente do volume de dados.

5.3 Validação em Língua Portuguesa

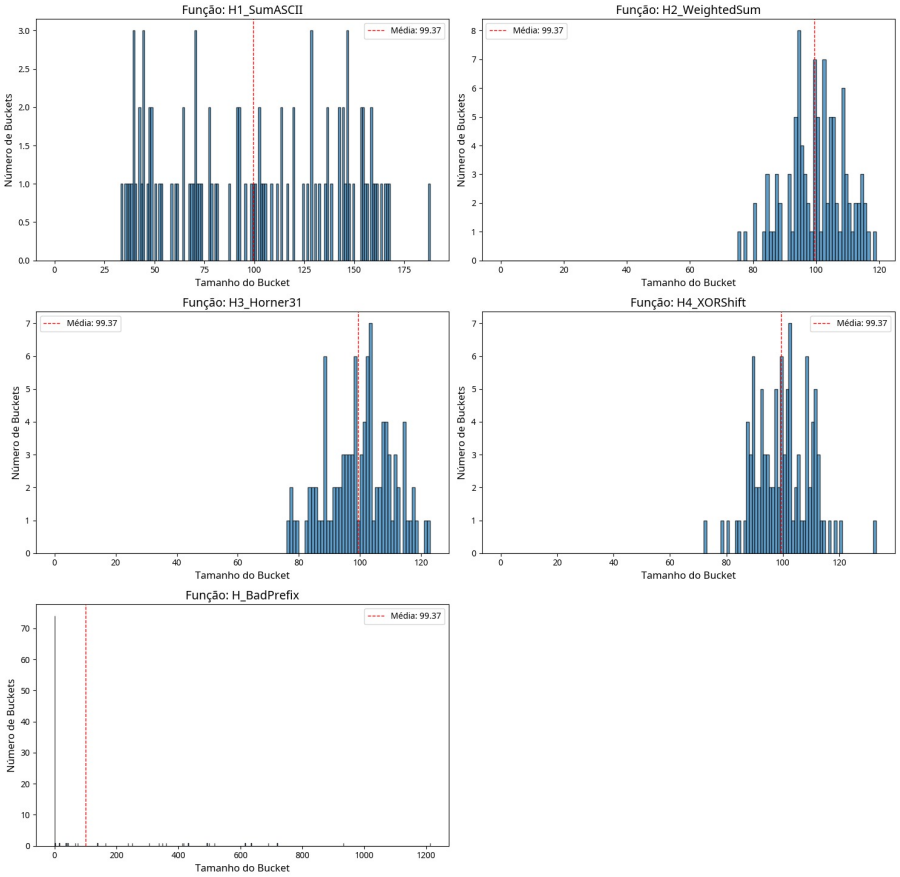

Figura 3: Distribuição para quincas_borba.txt com M=997.

Análise: A consistência dos histogramas com o texto de Machado de Assis valida a eficácia da nossa tokenização via regex, que capturou corretamente os acentos, e prova que a função Horner (H3) é universalmente eficiente.

6 Comparação: Hash vs. Linear

A eficiência de busca em uma tabela hash é exponencialmente superior à de uma estrutura linear.

Tabela 3: Comparação de esforço total (tale.txt, M=997)

Estrutura	**total_comp_success**	**total_comp_fail**
LinearBaseline	49.376.953	98.743.969
HashTable (H3)	59.960	99.343

Para a *LinearBaseline*, o custo é $O(n)$, exigindo milhões de comparações. Em contraste, a *HashTable* reduziu esse esforço em ordens de magnitude. Isso valida a superioridade das tabelas hash para operações em grandes volumes de dados.

7 Análise da Função 'Ruim' (H_BadPrefix)

A função **H_BadPrefix**, projetada intencionalmente para ser ineficiente (utilizando apenas o código ASCII do primeiro caractere), ilustra claramente as consequências de uma função hash mal elaborada.

Tabela 4: Desempenho da função ineficiente (tale.txt)

M	**max_bucket**	**avg_comp_success**	**avg_comp_fail**
97	1212	303.8178	547.9316
100	1212	303.8178	547.9316
997	1212	303.8178	547.9316

Independentemente do tamanho de M, a **H_BadPrefix** apresentou um *max_bucket* constante de 1212. Isso ocorre porque ela gera um número muito limitado de valores hash, concentrando a maioria das palavras em poucos buckets. Essa concentração transforma a busca em uma operação quase linear, degradando o desempenho para níveis piores que o baseline. Este experimento reforça a importância crítica de uma função que distribua uniformemente as chaves.

8 Conclusão

Este projeto demonstrou que a eficiência de uma tabela hash depende do equilíbrio entre uma função de espalhamento inteligente e um fator de carga gerenciado. A comparação final com a estrutura linear comprovou uma economia computacional assombrosa, enquanto a análise da função *BadPrefix* serviu como alerta para os riscos de implementações negligentes.